INF10025 Week 8 – Practice Answers

Model answers (Oracle SQL) with the expected result from the movie database in INF10025_Week08_Practice.accdb

All SQL answers were run against the same data as the Access file. Row counts and first rows are shown so you can check your own result. If your query returns the same rows in a different layout (column order or alias names), it is probably still correct.

Section A – Modelling: Many-to-many relationships, identifiers and relational schema

Q1. [Pass] A university lets each student enrol in many subjects, and each subject has many students. For every enrolment the system records the enrolment date and the final grade.
(a) Explain why the relational model cannot represent this many-to-many (M:M) relationship directly.
(b) Draw the ERD with the M:M expanded into two M:1 relationships.
(c) State the identifier of the intersection (weak) entity.
(d) Write the relational schema with PK and FK.

Model answer

(a) A relation (table) can only hold one value per attribute, so a student row cannot hold 'many subjects' and a subject row cannot hold 'many students'. The M:M must be replaced by two M:1 relationships.
(b) STUDENT (strong) --makes--> ENROLMENT (weak, double outline) <--for a-- SUBJECT (strong). The MANY end of both relationships attaches to ENROLMENT and both relationship names are underlined (identifying).
(c) ENROLMENT borrows the identifiers of both parents: StuId + SubCode (EnrolDate and FinalGrade are ordinary attributes).
(d) STUDENT (StuId, FirstName, Surname) PK: StuId
SUBJECT (SubCode, SubName) PK: SubCode
ENROLMENT (SubCode, StuId, EnrolDate, FinalGrade) PK: SubCode, StuId FK1: SubCode references SUBJECT FK2: StuId references STUDENT

Q2. [Credit] Glenferrie Toy Library records every loan of a toy to a patron (DateBorrowed, DateReturned). The LOAN table below has been created. Decide which rows are REJECTED for each candidate primary key.
LOAN rows (PatronNo, ToyNo, DateBorrowed, DateReturned):
1: 53, 27, 1 Jun, 8 Jun
2: 53, 31, 1 Jun, 8 Jun
3: 53, 27, 19 Jul, 30 Jul
4: 53, 27, 2 Aug, NULL
5: 71, 27, 11 Jun, 18 Jun
6: 53, 31, 9 Oct, NULL
(a) PK = (PatronNo, ToyNo)
(b) PK = (PatronNo, ToyNo, DateBorrowed)
(c) PK = (PatronNo, ToyNo, DateBorrowed, DateReturned)
Which one is the correct identifier and why?

Model answer

(a) Rows 3, 4 and 6 are rejected: each repeats an existing (PatronNo, ToyNo) pair (53,27 and 53,31). This identifier would not allow a patron to borrow the same toy twice.
(b) No row is rejected. Every row has a different combination of PatronNo + ToyNo + DateBorrowed.
(c) Rows 4 and 6 are rejected because DateReturned is NULL and a primary key must not contain NULLs (a toy that is still on loan has no return date).

The correct identifier is (b) PatronNo + ToyNo + DateBorrowed: a patron may borrow the same toy on different dates, and DateReturned must NOT be part of the identifier because it can be NULL.

Q3. [Credit] Hawthorn Fitness has members (MemberId, Name) and group classes (ClassId, ClassName, Weekday). A member can attend the same class on many different dates, and the system records how many minutes the member stayed. (a) Draw the ERD, (b) write the relational schema, and (c) create 4 sample rows of the weak entity that prove the identifier is correct.

Model answer

(a) MEMBER and CLASS are strong entities. The M:M 'attends' is expanded into ATTENDANCE (weak, double outline) with two underlined identifying relationships; Minutes is an attribute of ATTENDANCE.
(b) MEMBER (MemberId, Name) PK: MemberId
CLASS (ClassId, ClassName, Weekday) PK: ClassId
ATTENDANCE (MemberId, ClassId, AttendDate, Minutes) PK: MemberId, ClassId, AttendDate FK1: MemberId references MEMBER FK2: ClassId references CLASS
(c) (M01, C10, 2025-03-03, 55), (M01, C10, 2025-03-10, 60), (M01, C11, 2025-03-03, 45), (M02, C10, 2025-03-03, 50). Rows 1 and 2 have the same member and class but different dates, so they are only valid because AttendDate is part of the identifier.

Section B – Oracle date handling

Q4. [Pass] In Oracle, create a table EVENT (eventid NUMBER primary key, eventname VARCHAR(30), eventdate DATE). Insert one row using the DD-MON-YYYY format (21-MAY-2025) and one row using TO_DATE with the format dd/mm/yyyy (14/09/2025).

Model answer

```
CREATE TABLE EVENT (
  eventid   NUMBER PRIMARY KEY,
  eventname VARCHAR(30),
  eventdate DATE
);
INSERT INTO EVENT VALUES (1, 'Swim Carnival', '21-MAY-2025');
INSERT INTO EVENT VALUES (2, 'Open Day', TO_DATE('14/09/2025', 'dd/mm/yyyy'));
```

Q5. [Pass] List the full name and birth date of all actors born between 1 January 1970 and 31 December 1979 (inclusive), oldest first.

Model answer

```
SELECT fullname, birthdate
FROM   actor
WHERE  birthdate >= TO_DATE('01/01/1970', 'dd/mm/yyyy')
AND    birthdate <= TO_DATE('31/12/1979', 'dd/mm/yyyy')
ORDER BY birthdate;
```

Expected result: 146 row(s). First rows:

FullName	Birthdate
Paula Malcomson	1970-01-01
Noel Gugliemi	1970-01-01
Heather Graham	1970-01-29
Warwick Davis	1970-02-03

Q6. [Pass] In the toy library data, list PatronNo, ToyNo and DateBorrowed for every loan borrowed during June 2024.

Model answer

```
SELECT patronno, toyno, dateborrowed
FROM   loan
```

```
WHERE  dateborrowed BETWEEN TO_DATE('01/06/2024', 'dd/mm/yyyy')
                        AND TO_DATE('30/06/2024', 'dd/mm/yyyy')
ORDER BY dateborrowed, patronno;
```

Expected result: 4 row(s). First rows:

PatronNo	ToyNo	DateBorrowed
53	27	2024-06-01
53	31	2024-06-01
71	27	2024-06-11
71	31	2024-06-11

Section C – Subqueries

Q7. [Credit] Why does SELECT * FROM movie WHERE runtime > AVG(runtime); fail? Rewrite it so that it lists the title and runtime of every movie longer than the average, longest first.

Model answer

```
-- Aggregate functions are not allowed in WHERE; evaluate AVG in its own query (subquery).
SELECT title, runtime
FROM   movie
WHERE  runtime > (SELECT AVG(runtime) FROM movie)
ORDER BY runtime DESC, title;
```

Expected result: 170 row(s). First rows:

TITLE	RUNTIME
Hamlet	242
the oracle of swinburne	231
The Irishman	209
The Lord of the Rings: The Return of the King	201

Q8. [Credit] Find the title and runtime of the movie(s) with the longest running time.

Model answer

```
SELECT title, runtime
FROM   movie
WHERE  runtime = (SELECT MAX(runtime) FROM movie);
```

Expected result: 1 row(s). First rows:

TITLE	RUNTIME
Hamlet	242

Q9. [Credit] List the title and release year of all movies released in the same year as 'Forrest Gump', in alphabetical order.

Model answer

```
SELECT title, relyear
FROM   movie
WHERE  relyear = (SELECT relyear FROM movie WHERE title = 'Forrest Gump')
ORDER BY title;
```

Expected result: 8 row(s). First rows:

TITLE	RELYEAR
Dumb and Dumber	1994
Forrest Gump	1994
Four Weddings and a Funeral	1994
Muriel's Wedding	1994

Q10. [Credit] List the titles of all movies that have at least one actor born in Canada. Each title must appear only once.

Model answer

```
SELECT DISTINCT m.title
FROM   movie m
INNER JOIN casting c ON m.movieno = c.movieno
WHERE  c.actorno IN (SELECT actorno FROM actor WHERE birthcountry = 'Canada')
ORDER BY m.title;
```

Expected result: 80 row(s). First rows:

TITLE
50 First Dates
Ad Astra
Adventureland
Anchorman: The Legend of Ron Burgundy

Q11. [Credit] Which rating codes have an average runtime greater than the average runtime of all movies? Show the rating code and its average runtime (rounded to 1 decimal place).

Model answer

```
SELECT ratingcode, ROUND(AVG(runtime), 1) AS avg_runtime
FROM   movie
GROUP BY ratingcode
HAVING AVG(runtime) > (SELECT AVG(runtime) FROM movie)
ORDER BY ratingcode;
```

Expected result: 1 row(s). First rows:

RATINGCODE	avg_runtime
M	124.0

Q12. [Credit] List the title and TMDB score of all movies whose TMDB score is higher than the average TMDB score of movies rated 'PG'. Show the highest score first.

Model answer

```
SELECT title, tmdb_score
FROM   movie
WHERE  tmdb_score > (SELECT AVG(tmdb_score) FROM movie WHERE ratingcode = 'PG')
ORDER BY tmdb_score DESC, title;
```

Expected result: 213 row(s). First rows:

TITLE	TMDB_SCORE
Joker	8.8
The Irishman	8.6
The Shawshank Redemption	8.6
Avengers: Endgame	8.5

Q13. [Distinction] List the actor number and full name of every actor who has never been cast in any movie, using a subquery with NOT IN.

Model answer

```
SELECT actorno, fullname
FROM   actor
WHERE  actorno NOT IN (SELECT actorno FROM casting)
ORDER BY actorno;
```

Expected result: 3 row(s). First rows:

ActorNo	FullName
7	Andrew Stanton
8293	Marion Cotillard
90633	Gal Gadot

Q14. [Distinction] Use nested subqueries to list (alphabetically) the full names of all actors who appeared in the movie with the longest runtime.

Model answer

```
SELECT a.fullname
FROM   actor a
INNER JOIN casting c ON a.actorno = c.actorno
WHERE  c.movieno IN (SELECT movieno
                     FROM   movie
                     WHERE  runtime = (SELECT MAX(runtime) FROM movie))
ORDER BY a.fullname;
```

Expected result: 10 row(s). First rows:

FullName
Billy Crystal
Charlton Heston
Derek Jacobi
Gerard Depardieu

Section D – Inner and outer joins

Q15. [Pass] List the full name of every actor who appeared in the movie 'Forrest Gump', in alphabetical order (join ACTOR, CASTING and MOVIE).

Model answer

```
SELECT a.fullname
FROM   actor a
INNER JOIN casting c ON c.actorno = a.actorno
INNER JOIN movie   m ON m.movieno = c.movieno
WHERE  m.title = 'Forrest Gump'
ORDER BY a.fullname;
```

Expected result: 6 row(s). First rows:

FullName
Gary Sinise
Mary Ellen Trainor
Robin Wright
Sally Field

Q16. [Credit] Use a LEFT OUTER JOIN to list the actor number and full name of every actor who has no row in CASTING.

Model answer

```
SELECT a.actorno, a.fullname
FROM   actor a
LEFT OUTER JOIN casting c ON c.actorno = a.actorno
WHERE  c.castid IS NULL
ORDER BY a.actorno;
```

Expected result: 3 row(s). First rows:

ActorNo	FullName
7	Andrew Stanton
8293	Marion Cotillard
90633	Gal Gadot

Q17. [Credit] Rewrite your answer to the previous question using a RIGHT OUTER JOIN (CASTING must appear first in the FROM clause). The result must be identical.

Model answer

```
SELECT a.actorno, a.fullname
FROM   casting c
```

```
RIGHT OUTER JOIN actor a ON c.actorno = a.actorno
WHERE  c.castid IS NULL
ORDER BY a.actorno;
```

Expected result: 3 row(s). First rows:

ActorNo	FullName
7	Andrew Stanton
8293	Marion Cotillard
90633	Gal Gadot

Q18. [Credit] For every movie, show its number, title and the number of cast members, including movies that have no cast (count 0). Sort by cast size (smallest first) and then title.

Model answer

```
SELECT m.movieno, m.title, COUNT(c.castid) AS cast_size
FROM   movie m
LEFT OUTER JOIN casting c ON c.movieno = m.movieno
GROUP BY m.movieno, m.title
ORDER BY cast_size, m.title;
```

Expected result: 395 row(s). First rows:

MOVIENO	TITLE	cast_size
241259	Alice Through the Looking Glass	0
209112	Batman v Superman: Dawn of Justice	0
525	Blues Borthers	0
271110	Captain America: Civil War	0

Q19. [Credit] For the actors with ActorNo 3, 7, 8293 and 90633, list the full name, movie number and movie title of every movie they appeared in. Actors with no movies must still appear (title shown as NULL). Combine an INNER JOIN (casting and movie) with a RIGHT OUTER JOIN to actor.

Model answer

```
SELECT a.fullname, c.movieno, m.title
FROM   casting c
INNER JOIN movie m ON c.movieno = m.movieno
RIGHT OUTER JOIN actor a ON c.actorno = a.actorno
WHERE  a.actorno IN (3, 7, 8293, 90633)
ORDER BY a.fullname, m.title;
```

Expected result: 6 row(s). First rows:

FullName	MovieNo	TITLE
Andrew Stanton	NULL	NULL
Gal Gadot	NULL	NULL
Harrison Ford	78	Blade Runner
Harrison Ford	335984	Blade Runner 2049

Q20. [Credit] List every toy in the toy library with the surname of the patron who borrowed it and the date borrowed. Toys that have never been borrowed must also be listed. Sort by toy description, then date borrowed.

Model answer

```
SELECT t.description, p.surname, l.dateborrowed
FROM   toy t
LEFT OUTER JOIN loan   l ON l.toyno = t.toyno
LEFT OUTER JOIN patron p ON p.patronno = l.patronno
ORDER BY t.description, l.dateborrowed;
```

Expected result: 9 row(s). First rows:

Description	Surname	DateBorrowed
Building Blocks	NULL	NULL

Description	Surname	DateBorrowed
Jigsaw Puzzle	Dyson	2024-07-31
Rocking Horse	Vergara	2024-06-01
Rocking Horse	Dyson	2024-06-11

Q21. [Credit] List every student (id, first name, surname) with the number of subjects they have enrolled in, including students who have not enrolled in anything (count 0).

Model answer

```
SELECT s.stuid, s.firstname, s.surname, COUNT(e.subcode) AS subjects_enrolled
FROM   student s
LEFT OUTER JOIN enrolment e ON e.stuid = s.stuid
GROUP BY s.stuid, s.firstname, s.surname
ORDER BY s.stuid;
```

Expected result: 4 row(s). First rows:

StuId	FirstName	Surname	subjects_enrolled
1122334	Jane	Taylor	2
4444555	Jobe	McKay	2
7777700	Helen	Freeman	2
9900123	Ravi	Patel	0

Q22. [Distinction] Which subjects (code and name) have no enrolments at all? Answer twice: (a) with an outer join, (b) with a subquery.

Model answer

```
-- (a) outer join
SELECT s.subcode, s.subname
FROM   subject s
LEFT OUTER JOIN enrolment e ON e.subcode = s.subcode
WHERE  e.stuid IS NULL;

-- (b) subquery
SELECT subcode, subname
FROM   subject
WHERE  subcode NOT IN (SELECT subcode FROM enrolment);
```

Expected result (part 1): 1 row(s). First rows:

SubCode	SubName
INF10025	Data Management and Analytics

Expected result (part 2): 1 row(s). First rows:

SubCode	SubName
INF10025	Data Management and Analytics

Section E – UNION, UNION ALL, INTERSECT, MINUS and literals

Q23. [Credit] Two tables hold people: UNI_STUDENT(name) and UNI_TUTOR(name). (a) List all names that appear in either table, with no duplicates. (b) List all names from both tables, including duplicates. How many rows does (b) return and which name is repeated?

Model answer

```
-- (a)
SELECT name FROM uni_student
UNION
SELECT name FROM uni_tutor
ORDER BY name;

-- (b)
SELECT name FROM uni_student
UNION ALL
SELECT name FROM uni_tutor
```

```
ORDER BY name;
```

Expected result (part 1): 6 row(s). First rows:

Name
Amanda Allen
Bob Black
Carol Chen
Dave Dole

Expected result (part 2): 7 row(s). First rows:

Name
Amanda Allen
Bob Black
Carol Chen
Carol Chen

Q24. [Credit] List the names of people who are BOTH a student and a tutor (INTERSECT).

Model answer

```
SELECT name FROM uni_student
INTERSECT
SELECT name FROM uni_tutor;
```

Expected result: 1 row(s). First rows:

Name
Carol Chen

Q25. [Credit] (a) List the students who are not tutors. (b) List the tutors who are not students. Explain why the two answers differ (MINUS).

Model answer

```
-- (a)
SELECT name FROM uni_student
MINUS
SELECT name FROM uni_tutor
ORDER BY name;

-- (b)
SELECT name FROM uni_tutor
MINUS
SELECT name FROM uni_student
ORDER BY name;

-- MINUS returns the rows of the FIRST query that are not in the second, so swapping the queries changes the res
```

Expected result (part 1): 3 row(s). First rows:

Name
Amanda Allen
Bob Black
Dave Dole

Expected result (part 2): 2 row(s). First rows:

Name
Ed Ellis
Fred Foley

Q26. [Credit] List all customers with their gender. When the gender is NULL display the literal 'Unknown'. Use UNION of two queries, sorted by customer number.

Model answer

```
SELECT custnumb, name, gender
```

```
FROM   customer_w8
WHERE  gender IS NOT NULL
UNION
SELECT custnumb, name, 'Unknown'
FROM   customer_w8
WHERE  gender IS NULL
ORDER BY custnumb;
```

Expected result: 10 row(s). First rows:

CustNumb	Name	Gender
124	Celeste	F
256	Fred	M
311	Jeni	F
315	Louis	M

Q27. [Credit] List every customer number with the rating 'Good Customer' when balance <= credit limit, otherwise 'Bad Customer'. Use UNION and a literal column named Rating.

Model answer

```
SELECT custnumb, 'Good Customer' AS rating
FROM   customer_w8
WHERE  balance <= credlim
UNION
SELECT custnumb, 'Bad Customer' AS rating
FROM   customer_w8
WHERE  balance > credlim
ORDER BY custnumb;
```

Expected result: 10 row(s). First rows:

CustNumb	rating
124	Good Customer
256	Good Customer
311	Good Customer
315	Bad Customer

Q28. [Distinction] Produce one combined list with three columns Kind, Name and Yr containing (i) every movie released in 1994 (Kind = 'Movie', Name = title, Yr = release year) and (ii) every actor born in 1942 (Kind = 'Actor', Name = full name, Yr = birth year). The data types of the columns must match, so convert the numbers/dates with TO_CHAR.

Model answer

```
SELECT 'Movie' AS kind, title AS name, TO_CHAR(relyear) AS yr
FROM   movie
WHERE  relyear = 1994
UNION
SELECT 'Actor', fullname, TO_CHAR(birthdate, 'YYYY')
FROM   actor
WHERE  birthdate >= TO_DATE('01/01/1942', 'dd/mm/yyyy')
AND    birthdate <= TO_DATE('31/12/1942', 'dd/mm/yyyy')
ORDER BY kind, name;
```

Expected result: 12 row(s). First rows:

kind	name	yr
Actor	David Bradley	1942
Actor	Gemma Jones	1942
Actor	Harrison Ford	1942
Actor	Michael York	1942

Section F – Constraints: UNIQUE and CHECK

Q29. [Credit] Write the CREATE TABLE statement for EXAM with these rules: stuid NUMBER primary key; name VARCHAR(30) NOT NULL; gender VARCHAR(1) must be M, F or I; test1 must be >= 0; test2 must be between 0 and 50; test1 + test2 must never exceed 100; the combination (name, gender) must be unique.

Model answer

```
CREATE TABLE EXAM (
  stuid  NUMBER PRIMARY KEY
, name   VARCHAR(30) NOT NULL
, gender VARCHAR(1) CHECK (gender IN ('M', 'F', 'I'))
, test1  NUMBER CHECK (test1 >= 0)
, test2  NUMBER CHECK (test2 BETWEEN 0 AND 50)
, CHECK (test1 + test2 <= 100)
, UNIQUE (name, gender)
);
-- A UNIQUE constraint over several columns and a CHECK over several columns must be TABLE constraints (written
```

Q30. [Credit] Using the EXAM table from the previous question, state which of these INSERT rows succeed and which fail (and why):
1: (1,'Ann','F',60,40)
2: (2,'Bob','X',50,20)
3: (3,'Cy','M',-5,10)
4: (4,'Di','F',70,60)
5: (5,'Ed','M',NULL,NULL)
6: (6,'Flo','I',55,50)
7: (7,NULL,'F',10,10)
8: (1,'Gus','M',10,10)
9: (9,'Ann','F',20,20)

Model answer

1: succeeds.
2: fails - gender 'X' is not in ('M','F','I').
3: fails - test1 is negative (CHECK test1 >= 0).
4: fails - test2 = 60 is outside 0..50 (and test1 + test2 = 130 > 100).
5: succeeds - a CHECK passes when the condition is TRUE or NULL (unknown), so NULL scores pass.
6: fails - test1 + test2 = 105 > 100 (table-level CHECK).
7: fails - name is NOT NULL.
8: fails - stuid 1 already exists (primary key / unique violation).
9: fails - the pair ('Ann','F') duplicates row 1 (UNIQUE (name, gender)).

Q31. [Credit] For each proposed constraint say whether a CHECK constraint is suitable and explain: (a) CHECK (dayname IN ('Mon','Tue','Wed','Thu','Fri','Sat','Sun')); (b) CHECK (managername IN ('Dave','Sue','Henry')); (c) CHECK (enddate > SYSDATE); (d) CHECK (credlim >= (SELECT AVG(credlim) FROM customer)); (e) CHECK (grade IN ('Good','Excellent','Poor')).

Model answer

(a) Suitable: the seven day names are fixed and will never change.
(b) Not suitable: the list of managers changes over time; adding or removing a manager would require altering a table that may hold many rows. Use a separate MANAGER table with a foreign key.
(c) Not allowed: a CHECK cannot call functions such as SYSDATE (the value would change over time).
(d) Not allowed: a CHECK cannot contain a subquery or refer to values in other rows or tables.
(e) Not recommended: grades can be added or renamed; a lookup table with a foreign key is better.